MoMo SMS

Database Design Document

Week 2 — Database Foundation Assessment

Team: [EWD_Database Design and Implementation_Cohort 1_Team5

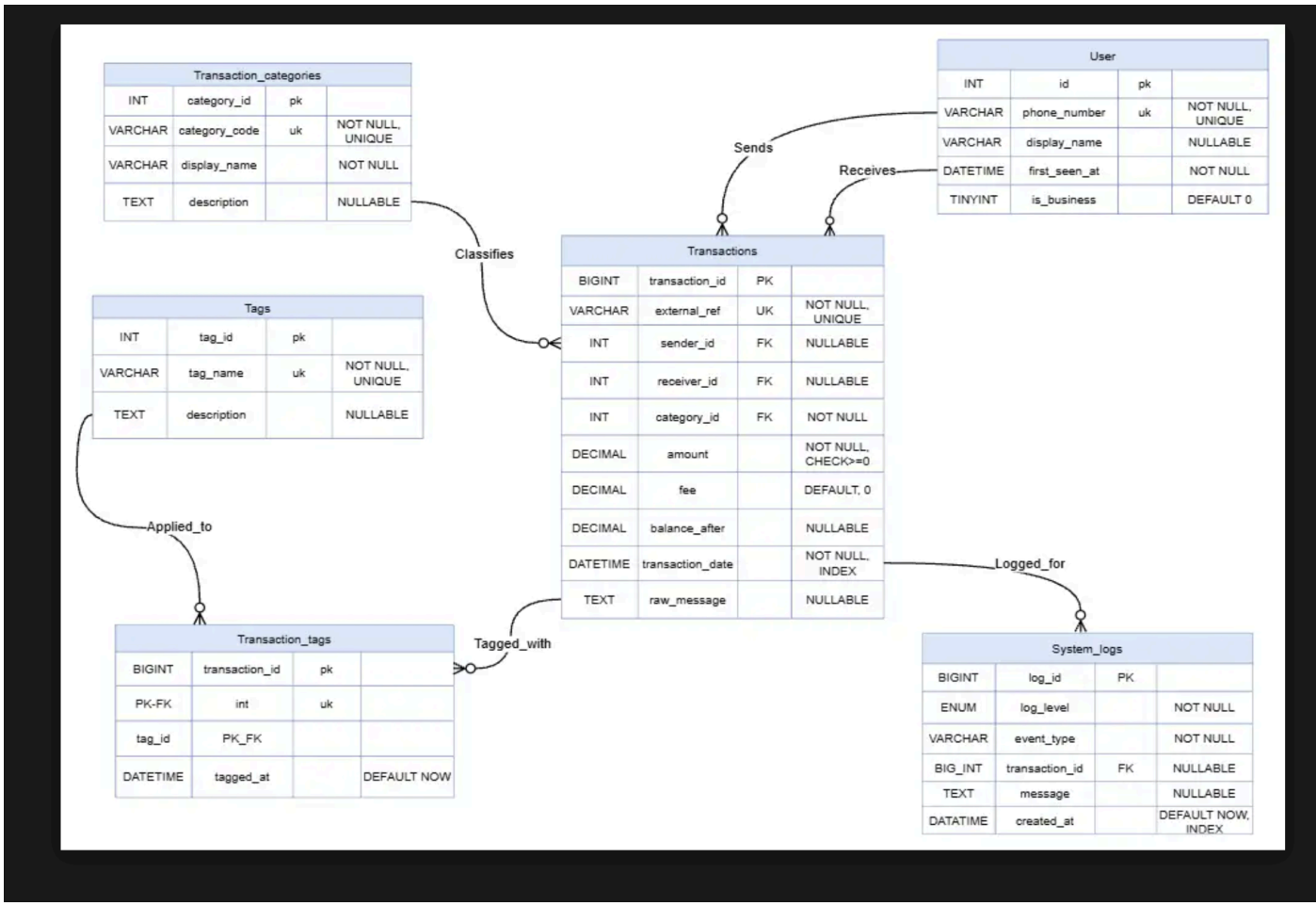
Members: Hassanat Bello, Panom Kot, Pegdwende Savadogo

Course: Enterprise Web Development

Institution: African Leadership University

ENTITY RELATIONSHIP DIAGRAM (ERD)

The ERD below shows all six entities, their attributes with data types, primary and foreign key markings, relationship cardinality (crow's-foot notation), and the junction table resolving the Transactions ↔ Tags many-to-many relationship.



DESIGN RATIONALE

The database schema for the MoMo SMS data processing system establishes the transactions table as the primary record, which receives support from the users, transaction_categories, system_logs, tags, and transaction_tags tables to improve data structure and system security and record system activities. The design establishes accurate data representation of the actual MoMo SMS system through its design, which enables future system expansion and database normalization and analytical efficiency and reporting efficiency. The first major design decision was to move customer information out of the transactions table and into the users table. The system needs an identity verification process which allows users to complete multiple transactions without repeating their personal information. The sender_id and receiver_id fields in the transactions table function as foreign keys that link to the users table. Some activities in MoMo require users to purchase airtime or balance their account or pay service fees without needing a second user, which is why the fields become nullable. The business process requires actual null values because they represent valid operational functions in business activities whereas phantom records need to be created for operational practices. The second critical choice was the handling of financial data. The system stores all monetary amounts as DECIMAL(15,2) because floating-point arithmetic leads to precision loss during calculations. Financial systems require absolute accuracy because they need to ensure that all financial balances and totals match exact cent amounts. The system uses a UNIQUE constraint with the transaction reference code to prevent the ETL pipeline from creating multiple transaction records when it processes the same XML file, which establishes essential data importability control.

The design process reached its third crucial decision when the team used the transaction_tags junction table to define how transactions relate to tags in a many-to-many connection. A single transaction can carry multiple tags such as "high-value" and "recurring" and "flagged-for-review" which apply to multiple transactions. The user-category M:N relationship did not meet our needs because each transaction in the source data belongs to one specific category which creates a true 1:M relationship. The tags relationship exists for analytical purposes because it enables dashboard queries which filter data based on tag combinations.

The database maintains data integrity through its CHECK (amount >= 0) system which requires essential fields to have NOT NULL values and enforces foreign key connections between all tables that relate to each other. The rules establish system entry restrictions which

stop invalid records from entering the system without regard to application-layer operations. The dashboard uses indexes on transaction_date and category_id to support its common access modes which include time-windowed aggregation and category-level breakdowns. The system_logs table stores ETL provenance data through its foreign key to transactions which allows it to capture both transaction-related and pipeline-related activities.

DATA DICTIONARY

Complete column-level reference for every table in the schema. AUTO_INC = AUTO_INCREMENT, CURRENT_TS = CURRENT_TIMESTAMP.

Table	Column	Type	Null	Default	Constraints	Description
users	user_id	INT	NO	AUTO_INC	PK	Surrogate primary key for user records
users	phone_number	VARCHAR(20)	NO	—	UNIQUE	MoMo account phone number in E.164 format
users	display_name	VARCHAR(100)	YES	NULL	—	Parsed name from SMS body, when available

Table	Column	Type	Null	Default	Constraints	Description
users	first_seen_at	DATETIME	NO	—	—	Timestamp of first transaction involving this user
users	is_business	TINYINT(1)	NO	0	—	Flag for business/merchant accounts
transactions	transaction_id	BIGINT	NO	AUTO_INC	PK	Surrogate primary key for transactions
transactions	external_ref	VARCHAR(100)	NO	—	UNIQUE	MoMo TxId from SMS; idempotency key
transactions	sender_id	INT	YES	NULL	FK→users	Sender user; null for non-P2P transactions
transactions	receiver_id	INT	YES	NULL	FK→users	Receiver user; null for airtime/bills
transactions	category_id	INT	NO	—	FK→categories	Transaction classification
transactions	amount	DECIMAL(15, 2)	NO	—	CHECK ≥0	Transaction amount in base currency (RWF)

Table	Column	Type	Null	Default	Constraints	Description
transactions	fee	DECIMAL(15, 2)	NO	0	—	MoMo fee charged on transaction
transactions	balance_after	DECIMAL(15, 2)	YES	NULL	—	Account balance after this transaction
transactions	transaction_date	DATETIME	NO	—	INDEXED	When the transaction occurred
transactions	raw_message	TEXT	YES	NULL	—	Original SMS body for audit trail
transaction_categories	category_id	INT	NO	AUTO_INC	PK	Surrogate primary key for categories
transaction_categories	category_code	VARCHAR(50)	NO	—	UNIQUE	Stable code (e.g., TRANSFER_OUT)
transaction_categories	display_name	VARCHAR(100)	NO	—	—	Human-readable category name
transaction_categories	description	TEXT	YES	NULL	—	Category description for documentation

Table	Column	Type	Null	Default	Constraints	Description
tags	tag_id	INT	NO	AUTO_INC	PK	Surrogate primary key for tags
tags	tag_name	VARCHAR(50)	NO	—	UNIQUE	Tag identifier (e.g., high_value)
tags	description	TEXT	YES	NULL	—	Tag purpose and criteria
transaction_tags	transaction_id	BIGINT	NO	—	PK, FK→transactions	Composite PK part 1
transaction_tags	tag_id	INT	NO	—	PK, FK→tags	Composite PK part 2
transaction_tags	tagged_at	DATETIME	NO	CURRENT_TS	—	When tag was applied
system_logs	log_id	BIGINT	NO	AUTO_INC	PK	Surrogate primary key for log entries
system_logs	log_level	ENUM	NO	—	—	INFO, WARN, or ERROR
system_logs	event_type	VARCHAR(50)	NO	—	—	Event identifier (e.g., PARSE_FAILED)

Table	Column	Type	Null	Default	Constraints	Description
system_logs	transaction_id	BIGINT	YES	NULL	FK→transactions	Linked transaction; null for global events
system_logs	message	TEXT	YES	NULL	—	Free-form log message
system_logs	created_at	DATETIME	NO	CURRENT_TS	INDEXED	When the event was logged

Testing and Validation

Database functionality was validated in six phases: schema verification, constraint enforcement, CRUD operations, analytical queries, JSON validation, and clean-environment regression testing

CONSTRAINTS ENFORCEMENTS TESTS

Four negative test cases confirm that database-level constraints are enforced. CHECK prevents negative amounts; UNIQUE prevents duplicate transaction references (ETL idempotency); FOREIGN KEY prevents orphan sender references; NOT NULL prevents missing category classifications.

```
hassanah@Hassanats-MacBook-Pro Momo-sms-analytics % mysql -u root -p momo_sms
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> INSERT INTO transactions (external_ref, category_id, amount, transaction_date)
    -> VALUES ('TEST-NEG', 1, -100.00, NOW());
ERROR 3819 (HY000): Check constraint 'chk_transactions_amount_nonneg' is violated.
mysql> INSERT INTO transactions (external_ref, category_id, amount, transaction_date)
    -> VALUES ('TXN20260401001', 1, 1000.00, NOW());
ERROR 1062 (23000): Duplicate entry 'TXN20260401001' for key 'transactions.uk_transactions_external_ref'
mysql> INSERT INTO transactions (external_ref, sender_id, category_id, amount, transaction_date)
    -> VALUES ('TEST-FK', 99999, 1, 1000.00, NOW());
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`momo_sms`.`transactions`, CONSTRAINT `fk_transa
ctions_sender` FOREIGN KEY (`sender_id`) REFERENCES `users` (`user_id`))
mysql> INSERT INTO transactions (external_ref, amount, transaction_date)
    -> VALUES ('TEST-NULL', 1000.00, NOW());
ERROR 1364 (HY000): Field 'category_id' doesn't have a default value
mysql> exit
Bye
hassanah@Hassanats-MacBook-Pro Momo-sms-analytics %
```

CRUD Operations

Standard create/read/update/delete operations were tested against the users and transactions tables. The READ (join) test exercises the relational integrity of the schema by joining sender, receiver, category, and transaction data in a single query.

Operation	SQL (abbreviated)	Expected Result
CREATE	INSERT INTO users	Inserts new user; returns LAST_INSERT_ID()
READ (basic)	SELECT * FROM users WHERE ...	Returns the inserted row
READ (join)	Multi-table JOIN on transactions	Returns 5 transactions with sender/receiver/category resolved
UPDATE	UPDATE users SET display_name	display_name changed in subsequent SELECT
DELETE	DELETE FROM users WHERE ...	Row removed; subsequent SELECT returns 0 rows

C

```
mysql> INSERT INTO users (phone_number, display_name, first_seen_at, is_business)
    -> VALUES ('+250788999111', 'Test User', NOW(), 0);
Query OK, 1 row affected (0.094 sec)

mysql> SELECT LAST_INSERT_ID();
+------------------+
| LAST_INSERT_ID() |
+------------------+
|                9 |
+------------------+
1 row in set (0.007 sec)
```

R

```
mysql> SELECT * FROM users WHERE phone_number = '+250788999111';
+---------+---------------+--------------+---------------------+-------------+
| user_id | phone_number  | display_name | first_seen_at       | is_business |
+---------+---------------+--------------+---------------------+-------------+
|       9 | +250788999111 | Test User    | 2026-05-14 22:37:32 |           0 |
+---------+---------------+--------------+---------------------+-------------+
1 row in set (0.009 sec)
```

R-join

```
mysql> SELECT
    ->     t.transaction_id,
    ->     t.amount,
    ->     s.display_name AS sender,
    ->     r.display_name AS receiver,
    ->     c.display_name AS category
    -> FROM transactions t
    -> LEFT JOIN users s ON t.sender_id = s.user_id
    -> LEFT JOIN users r ON t.receiver_id = r.user_id
    -> JOIN transaction_categories c ON t.category_id = c.category_id
    -> LIMIT 5;
+----------------+----------+---------------+----------------+---------------------------+
| transaction_id | amount   | sender        | receiver       | category                  |
+----------------+----------+---------------+----------------+---------------------------+
|              1 |  5000.00 | Panom Kot     | Aline Uwase    | Person-to-Person Transfer |
|              7 |  3000.00 | Eric Habimana | Patrick Nshuti | Person-to-Person Transfer |
|              9 |  7000.00 | Aline Uwase   | Eric Habimana  | Person-to-Person Transfer |
|              2 |  1000.00 | Aline Uwase   | NULL           | Airtime Purchase          |
|              5 | 20000.00 | Panom Kot     | NULL           | Cash Withdrawal           |
+----------------+----------+---------------+----------------+---------------------------+
5 rows in set (0.037 sec)
```

U

```
mysql> UPDATE users SET display_name = 'Updated Name'
    -> WHERE phone_number = '+250788999111';
Query OK, 1 row affected (0.040 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT * FROM users WHERE phone_number = '+250788999111';
+---------+---------------+--------------+---------------------+-------------+
| user_id | phone_number  | display_name | first_seen_at       | is_business |
+---------+---------------+--------------+---------------------+-------------+
|       9 | +250788999111 | Updated Name | 2026-05-14 22:37:32 |           0 |
+---------+---------------+--------------+---------------------+-------------+
1 row in set (0.001 sec)
```

D

```
mysql>
mysql> -- DELETE
Query OK, 0 rows affected (0.000 sec)

mysql> DELETE FROM users WHERE phone_number = '+250788999111';
Query OK, 1 row affected (0.002 sec)

mysql> SELECT COUNT(*) FROM users WHERE phone_number = '+250788999111';
+----------+
| COUNT(*) |
+----------+
|        0 |
+----------+
1 row in set (0.001 sec)

mysql>
```

Analytical Queries

Five queries were run to validate the schema supports realistic dashboard and operational workloads. Query 4 specifically exercises the M:N junction table, confirming the design choice in Section 3 earns its place.

#	Query Purpose	Business Question Answered
Q1	Top categories by total volume	Dashboard: which categories drive the most money?
Q2	Monthly transaction count and average amount	Dashboard: trend analysis over time
Q3	Top 20 senders by transaction count	Dashboard: power users / business accounts
Q4	Transactions with multiple specific tags (M:N junction)	Demonstrates junction table earns its place; finds high_value + recurring transactions
Q5	Failed/warned ETL events grouped by type	Operations: ETL health monitoring

Q1: Top categories by total volume (dashboard query)

```
mysql> SELECT c.display_name, COUNT(*) AS txn_count, SUM(t.amount) AS total_amount
    -> FROM transactions t
    -> JOIN transaction_categories c ON t.category_id = c.category_id
    -> GROUP BY c.category_id, c.display_name
    -> ORDER BY total_amount DESC
    -> LIMIT 10;
+---------------------------+-----------+--------------+
| display_name              | txn_count | total_amount |
+---------------------------+-----------+--------------+
| Merchant Payment          |         3 |    350000.00 |
| Cash Deposit              |         1 |     50000.00 |
| Cash Withdrawal           |         1 |     20000.00 |
| Person-to-Person Transfer |         3 |     15000.00 |
| Bill Payment              |         1 |      8500.00 |
| Airtime Purchase          |         1 |      1000.00 |
+---------------------------+-----------+--------------+
6 rows in set (0.080 sec)
```

Q2: Monthly aggregation

```
mysql> SELECT
    ->     DATE_FORMAT(transaction_date, '%Y-%m') AS month,
    ->     COUNT(*) AS txn_count,
    ->     AVG(amount) AS avg_amount,
    ->     SUM(amount) AS total_amount
    -> FROM transactions
    -> GROUP BY DATE_FORMAT(transaction_date, '%Y-%m')
    -> ORDER BY month DESC;
+---------+-----------+--------------+--------------+
| month   | txn_count | avg_amount   | total_amount |
+---------+-----------+--------------+--------------+
| 2026-04 |        10 | 44450.000000 |    444500.00 |
+---------+-----------+--------------+--------------+
1 row in set (0.015 sec)
```

Q3: Top active users with most-used category

```
mysql> SELECT
    ->     u.display_name,
    ->     u.phone_number,
    ->     COUNT(t.transaction_id) AS txn_count,
    ->     SUM(t.amount) AS total_sent
    -> FROM users u
    -> JOIN transactions t ON u.user_id = t.sender_id
    -> GROUP BY u.user_id
    -> ORDER BY txn_count DESC
    -> LIMIT 20;
+------------------+---------------+-----------+------------+
| display_name     | phone_number  | txn_count | total_sent |
+------------------+---------------+-----------+------------+
| Panom Kot        | +250788111001 |         2 |   25000.00 |
| Aline Uwase      | +250788222002 |         2 |    8000.00 |
| Chantal Ingabire | +250788666006 |         2 |  325000.00 |
| Patrick Nshuti   | +250788333003 |         1 |   25000.00 |
| Eric Habimana    | +250788777007 |         1 |    3000.00 |
| Anonymous User   | +250788888008 |         1 |    8500.00 |
+------------------+---------------+-----------+------------+
6 rows in set (0.006 sec)
```

Q4: Demonstrates the M:N junction works (transactions with specific tag combinations)

```
mysql> SELECT t.transaction_id, t.amount, GROUP_CONCAT(tg.tag_name) AS tags
    -> FROM transactions t
    -> JOIN transaction_tags tt ON t.transaction_id = tt.transaction_id
    -> JOIN tags tg ON tt.tag_id = tg.tag_id
    -> WHERE t.transaction_id IN (
    ->     SELECT transaction_id FROM transaction_tags tt2
    ->     JOIN tags tg2 ON tt2.tag_id = tg2.tag_id
    ->     WHERE tg2.tag_name IN ('high_value', 'recurring')
    ->     GROUP BY transaction_id
    ->     HAVING COUNT(DISTINCT tg2.tag_name) = 2
    -> )
    -> GROUP BY t.transaction_id;
+----------------+-----------+----------------------------------+
| transaction_id | amount    | tags                             |
+----------------+-----------+----------------------------------+
|              6 | 150000.00 | high_value,recurring             |
|             10 | 175000.00 | after_hours,high_value,recurring |
+----------------+-----------+----------------------------------+
2 rows in set (0.058 sec)
```

Q5: ETL idempotency check (proves duplicate detection via system_logs)

```
mysql> SELECT event_type, COUNT(*) AS occurrences
    -> FROM system_logs
    -> WHERE log_level IN ('WARN', 'ERROR')
    -> GROUP BY event_type
    -> ORDER BY occurrences DESC;
+------------------+--------------+
| event_type       | occurrences |
+------------------+--------------+
| MISSING_BALANCE  |            1 |
| DUPLICATE_REF    |            1 |
| PARSE_FAIL       |            1 |
+------------------+--------------+
3 rows in set (0.005 sec)
```

JSON Validation

All JSON files in /examples were validated for syntactic correctness using python -m json.tool. IDs and values across the JSON examples are consistent with the seed data inserted by database_setup.sql, ensuring the JSON examples are not detached from the actual schema.

```
hassanah@Hassanats-MacBook-Pro Momo-sms-analytics % for f in examples/*.json; do
    echo "Checking $f"
    python3 -m json.tool "$f" > /dev/null && echo "  OK" || echo "  INVALID"
done
Checking examples/dashboard_aggregate.json
  OK
Checking examples/system_log.json
  OK
Checking examples/tag.json
  OK
Checking examples/transaction.json
  OK
Checking examples/transaction_category.json
  OK
Checking examples/transaction_full.json
  OK
Checking examples/user.json
  OK
hassanah@Hassanats-MacBook-Pro Momo-sms-analytics %
```

```
hassanah@Hassanats-MacBook-Pro Momo-sms-analytics % grep -A2 'user_id' examples/user.json
grep "INSERT INTO users" database/database_setup.sql | head -5
  "user_id": 42,
  "phone_number": "+250788123456",
  "display_name": "Jean Bizimana",
INSERT INTO users (phone_number, display_name, first_seen_at, is_business) VALUES
hassanah@Hassanats-MacBook-Pro Momo-sms-analytics %
```